

UpsideFuzzer: Coverage-Guided REST API Fuzzing for .NET via Semantic Source-Aware Grammar Enhancement

Sergei Ovchinnikov
sergey.o.a@gmail.com

Abstract—REST APIs are hard to fuzz well because good inputs must be valid at several levels at once. The request must match the HTTP route, the JSON body must be well formed, and the field values must pass the checks used by the application itself. Tools such as RESTler [1] solve the request-sequence problem, but they still use generic values that are often rejected by server-side validation. Standard coverage-guided fuzzers have the opposite problem: they get useful feedback from code coverage, but byte-level mutation breaks HTTP and JSON too easily.

We present UpsideFuzzer, a greybox fuzzing framework for .NET REST services. It has three main parts. First, *Semantic Source Extraction* (SSE) reads C# validation rules from Data Annotations and FluentValidation and adds better boundary values to RESTler-compatible grammar files. Second, an automated instrumentation pipeline adds SharpFuzz [3] coverage hooks to Dockerized .NET services. Third, the *Void* runtime sends typed requests from the grammar, tracks new coverage, learns IDs and tokens from successful responses, and uses that information to choose future requests and mutations.

On a 24-hour evaluation over three .NET applications, UpsideFuzzer covers 127% to 169% more unique edges than the RESTler baseline and reaches the first crash much earlier on the tested targets. We also ran a reproducible 20-minute Docker benchmark on four public .NET services. In that benchmark, UpsideFuzzer shows equal or better OpenAPI coverage, much higher throughput, and more HTTP 500-producing method+endpoint pairs overall than the RESTler baseline.

Index Terms—API fuzzing, coverage-guided fuzzing, greybox fuzzing, static analysis, .NET, REST API, instrumentation, OpenAPI

I. INTRODUCTION

REST APIs are a common attack surface, but they are harder to fuzz than ordinary byte-oriented programs. To reach interesting code, a request usually has to be correct at several levels: the route and method must match, the body must have valid syntax, and the values inside that body must satisfy the rules used by the application. If any of these checks fail, the request is rejected early and the fuzzer learns little about deeper program logic.

Stateful black-box API fuzzers such as RESTler [1] handle the first part of this problem well. Starting from an OpenAPI specification, they infer producer-consumer dependencies and generate valid request sequences. Their weak point is the value space. Default values such as "fuzzstring" or 0 often fail server-side validation, enum checks, or business rules before the request reaches deeper logic. As Godefroid et al. later showed [2], the values inside API requests are an important part of the search space.

Coverage-guided fuzzers solve a different problem: they use code coverage to guide the search toward new behavior. But most of them mutate raw bytes. For REST APIs, that is usually too destructive. Small byte changes can break the HTTP envelope, the JSON structure, or the path format, so many generated inputs fail before they reach the application code.

UpsideFuzzer combines these two ideas in a simpler way: it keeps RESTler's grammar and request sequencing, but it replaces generic values with better ones taken from the source code and uses coverage feedback to decide what to try next. The key point is that UpsideFuzzer mutates *typed grammar slots*, not arbitrary bytes. Integers stay integers, UUIDs keep their shape, JSON stays parseable, and IDs learned from earlier responses can be reused in later requests. This makes the coverage signal much more useful, because it can be tied to concrete choices such as which endpoint was chosen, which field value was changed, and which runtime dependency was replayed.

Contributions. This paper makes the following contributions:

- **Semantic Source Extraction (SSE):** A static analysis technique for C# that recovers field-level validation constraints from Data Annotation attributes and FluentValidation rules and synthesizes boundary-focused dictionary values for RESTler's grammar without modifying the RESTler compiler (§V).
- **Executable Grammar Materialization:** A runtime-oriented grammar export that converts RESTler compiler output into typed templates, dependency descriptors, and dictionary-backed placeholders that can be scheduled, mutated, and attributed by a greybox engine (§III-A).
- **Multipart Grammar Injection:** An automated pass that detects multipart/form-data endpoints in OpenAPI specifications and injects syntactically correct, RESTler-native seed request templates, enabling fuzzing of file-upload endpoints that RESTler's compiler cannot handle out-of-the-box (§VI).
- **Automated IL Instrumentation Pipeline:** A reproducible pipeline that instruments arbitrary .NET Docker images via SharpFuzz without requiring manual harness development, by patching the application Dockerfile at build time (§IV).
- **Void Execution Engine and Feedback Loop:** A highly

concurrent Go-based runner that drives the greybox evolutionary loop over RESTler-compiled grammars, combining structured mutation, runtime value learning, endpoint-level prioritization, and shared-memory coverage feedback from the instrumented .NET process (§VII).

- **Empirical Evaluation:** A 24-hour comparative study on three .NET applications, plus a reproducible Docker benchmark on four public services, comparing coverage, throughput, and crash discovery against the RESTler black-box baseline (§VIII).

II. BACKGROUND AND RELATED WORK

A. Coverage-Guided Greybox Fuzzing

Coverage-guided greybox fuzzing uses program coverage to decide which inputs are worth keeping. In AFL [4], instrumentation adds counters to branch edges, a shared-memory bitmap records which edges were hit, and any input that reaches new coverage is added to the corpus. SharpFuzz [3] brings the same basic idea to .NET by rewriting compiled IL and inserting edge counters without requiring source-code changes.

B. Specification-Based API Fuzzing

RESTler [1] is a stateful REST API fuzzer built around OpenAPI specifications. It analyzes producer-consumer dependencies between operations, then generates request sequences that follow those dependencies. Its grammar separates *static* tokens copied from the specification from *fuzzable* tokens that are filled with values at runtime. This works well for request sequencing and basic structure, but the default value dictionaries are small and generic, so domain-specific values usually have to be added by hand.

C. Intelligent API Data Fuzzing

Godofroid et al. [2] later focused on the values inside REST requests, not just the request sequence. They explored schema-level body mutations such as type substitution, field deletion, and value replication, and also studied two ways to get better concrete values: using examples from the specification and learning values from earlier responses. Their results show that generic random values are weak for structured JSON and that better values improve bug discovery. UpsideFuzzer follows the same general direction, but gets these values from .NET source code rather than only from the specification or earlier responses.

D. Other Related Tools

EvoMaster [5] is a white-box REST API testing tool for JVM services. It uses evolutionary search and runtime instrumentation, but it targets a different ecosystem and does not recover validation rules from source annotations. WuppieFuzz [6] is a newer framework built on LibAFL that supports several API fuzzing modes, but it also does not extract validation semantics from application source code. Dredd [7], APIFuzzer [8], and schemathesis [9] generate tests directly from API specifications and do not use code coverage. In short, existing tools usually give you either good request

structure or good coverage guidance; UpsideFuzzer is meant to combine those two with source-derived validation values for .NET services.

III. SYSTEM ARCHITECTURE

UpsideFuzzer has four main components and runs in two phases: an offline *preparation phase* and an online *fuzzing phase*. Figure 1 shows the full pipeline.

Preparation phase. Given a target .NET application repository and an OpenAPI specification, the user runs:

- 1) `fuzz-prep-multi.py`: instruments the application image (Section IV);
- 2) `compile-grammar.sh`: invokes the RESTler compiler to produce a raw grammar, then calls the SSE enhancer (Section V) to augment it.

Fuzzing phase. The Void runtime (Section VII) runs next to the instrumented application container. It sends requests from the grammar, reads coverage from the target, and updates its seed corpus over time.

A. Grammar Artifacts and Runtime Input Model

The online fuzzer does not read raw OpenAPI JSON directly. Instead, `compile-grammar.sh` produces three files that Void uses at runtime:

- 1) **grammar.py**: the original RESTler grammar, preserved as the canonical compiler output;
- 2) **dict.json**: the mutation dictionary, augmented by SSE with domain-specific boundary values, enum members, and format-aware examples;
- 3) **templates.export.json**: a normalized runtime view of the grammar in which every RESTler Request object is exported into a compact, language-agnostic representation that Void can load directly.

Each exported template corresponds to one OpenAPI operation. It contains a `request_id`, a list of `segments`, and `reads/writes` dependency annotations recovered by RESTler. A segment can be one of three kinds:

- **static**: immutable bytes copied verbatim from the grammar, such as HTTP verbs, route delimiters, header names, or fixed query keys;
- **fuzzable**: a typed placeholder (e.g., `string`, `int`, `uuid`, `datetime`) whose runtime value is chosen from `dict.json`, optionally mutated further by Void;
- **custom_payload**: a dependency-bearing placeholder whose value must be learned dynamically from earlier successful responses (e.g., a server-generated resource identifier).

This format matters for two reasons. First, requests stay structurally valid, because mutation only changes typed slots instead of editing random bytes in a serialized HTTP message. Second, the `reads/writes` annotations give Void enough information to handle state, for example when one request creates an ID that another request must reuse later. In practice, the grammar becomes an executable input model for valid and stateful HTTP requests.

B. Lowering an Endpoint into a Runtime Template

The endpoint-to-grammar lowering step is where the OpenAPI description turns into something the greybox runtime can use. A single operation such as

```
POST /api/orders/{orderId}/lines?api-version=1.0
Content-Type: application/json

{"sku": "...", "quantity": 1}
```

is exported into a template that keeps the HTTP skeleton but exposes only the useful choice points:

```
request_id: post_order_line
segments:
- static: "POST /api/orders/"
- custom_payload: orderId
- static: "/lines?api-version=1.0 HTTP/1.1\r\n..."
- static: "{\"sku\":\""
- fuzzable(string): sku
- static: "\",\"quantity\":\""
- fuzzable(int): quantity
- static: "\""
reads: ["orderId"]
writes: []
```

This makes the control points explicit. Path parameters that depend on earlier requests become `custom_payload` slots, fixed query values stay `static`, and body fields become typed `fuzzable` slots. Void does not try to discover the route shape by byte mutation. It inherits the route and framing from the template and only searches over the values inserted into the slots. If an earlier response produced an order identifier, the `orderId` slot is filled from the runtime store; otherwise Void falls back to the dictionary and then mutates the value in a type-aware way.

This template form also makes later coverage attribution possible. Each rendered request keeps its `request_id`, HTTP method, normalized path, and mutation label, so a later edge gain can be tied back to a concrete operation and mutation choice.

IV. AUTOMATED INSTRUMENTATION PIPELINE

To collect edge coverage without writing a custom harness, UpsideFuzzer adds SharpFuzz instrumentation during the Docker build. The preparation script `fuzz-prep-multi.py` does the following:

- 1) **Clone and patch the build.** The script clones the target repository and adds the SharpFuzz.Instrumentation package to the Docker build.
- 2) **Rewrite IL.** During the image build, SharpFuzz rewrites the target `.dll` files and inserts an edge counter before each branch target:

$$\text{bitmap}[(\text{prev_loc} \oplus \text{cur_loc}) \gg 1] += 1 \quad (1)$$

where `prev_loc` and `cur_loc` are random 16-bit identifiers for basic blocks, following the AFL shared-memory protocol [4]. This produces a 65,536-byte bitmap in shared memory.

- 3) **Expose coverage over HTTP when needed.** The script can patch `Program.cs` to expose the shared-memory

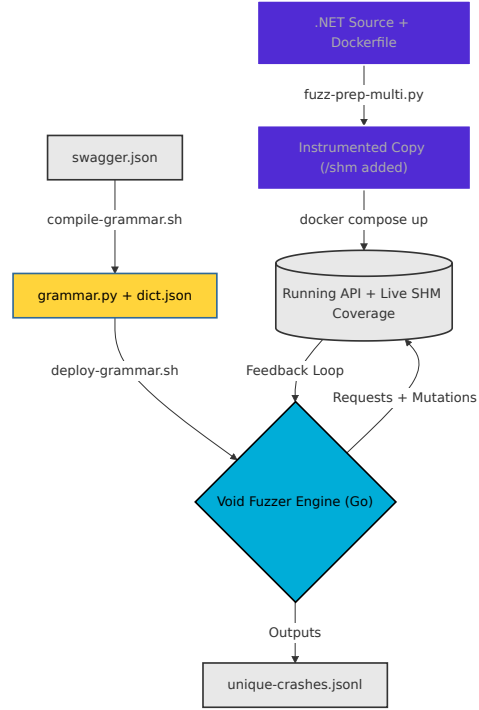


Fig. 1. UpsideFuzzer Architecture and End-to-End Pipeline.

bitmap through HTTP, which is useful in Docker setups where direct file sharing is inconvenient.

- 4) **Update Docker Compose.** The compose file is updated so the coverage endpoint and shared-memory region are available at runtime.

In practice, this adds only one extra Docker build cycle and does not require any manual harness code.

V. SEMANTIC SOURCE EXTRACTION

A. Motivation

RESTler knows how to build valid request sequences, but its default values are often too generic to pass application-level validation. For example, if a backend requires `[StringLength(8, MinimumLength=4)]`, a value such as `"fuzzstring"` may be rejected immediately. In that case the request never reaches the deeper logic behind the validator. Semantic Source Extraction (SSE) tries to fix this by filling the dictionary with better boundary values taken from the application's own validation rules.

B. Pipeline Overview

SSE has two parts. First, it reads the OpenAPI specification and collects constraints such as `minLength`, `maxLength`, `minimum`, `maximum`, `pattern`, and `enum`. Second, it scans the C# source tree to recover constraints that are missing from

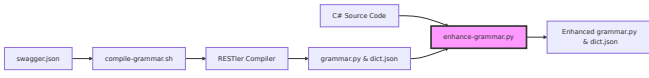


Fig. 2. Semantic Source-Aware Grammar Enhancement Pipeline.

the specification or more precise than what the specification says.

C. OpenAPI Constraint Extraction

The `OpenAPIExtractor` walks through the Swagger schema, resolves `$ref` links, and expands `allOf/oneOf/anyOf` nodes. For each final property, it builds a `FieldHint` record containing the field type, format, declared constraints, and any inline `example` or `default` value. If a field is a binary string in a `multipart/form-data` operation, it is marked for the multipart injection pass (Section VI).

D. C# Source Code Constraint Extraction

The `SourceExtractor` runs a regex-based pass over all `.cs` files in the target tree. A full Roslyn-based analysis would be more precise, but regex is much lighter and works well for the common validation patterns used in .NET projects.

Data Annotation Attributes. The extractor identifies attribute blocks preceding property or parameter declarations and parses the following attributes (case-insensitive):

- `[StringLength(N)]` and `[StringLength(N, MinimumLength=M)]`: sets `max_length = N`, `min_length = M`.
- `[MaxLength(N)]`, `[MinLength(M)]`: sets the corresponding bound.

- `[Range(lo, hi)]`: sets `minimum = lo`, `maximum = hi`.
- `[RegularExpression("pattern")]`: stores the regex pattern.
- `[Required]`: marks the field as non-nullable.
- `[EmailAddress]`, `[Url]`: activates format-specific value generators.

FluentValidation Rules. For validators that extend `AbstractValidator<T>`, the extractor scans `RuleFor(...)` chains and recognizes the following rule methods:

- `.MaxLength(N)`, `.MinimumLength(M)`, `.Length(M,N)`: string length bounds.
- `.InclusiveBetween(lo, hi)`, `.ExclusiveBetween(lo, hi)`: numeric bounds.
- `.GreaterThan(v)`, `.LessThan(v)`: one-sided numeric bounds.
- `.Matches("pattern")`: regex pattern.
- `.NotEmpty()`, `.NotNull()`: required-field marking.
- `.EmailAddress()`, `.Must(IsUrl)`: format validators.

When the same constraint appears in both the specification and the source code, SSE keeps the tighter one.

Enum Resolution. For each property or parameter whose type resolves to a C# enum, the extractor locates the enum declaration in the source tree and parses all symbolic members, including their optional explicit integer assignments. Both the symbolic name and its integer equivalent are added to the dictionary, following the observation in [2] that enum-constrained fields require exact value matches to progress past validation.

E. Boundary Value Synthesis

Given a `FieldHint`, SSE builds a small set of boundary-focused test values. For numeric fields it uses $\{lo - 1, lo, hi, hi + 1\}$. For string-length constraints it uses values around the minimum and maximum lengths. For regex constraints it adds one matching and one non-matching example. These values are inserted into the RESTler dictionary under several name variants such as `camelCase`, `PascalCase`, and `lowercase`.

An important point is that SSE does not replace RESTler's grammar or compiler. It only adds better values to the dictionary and, when needed, appends extra seed templates.

F. From Field Hints to Grammar Augmentation

The SSE output is merged back into the grammar in a format RESTler already understands. UpsideFuzzer does not introduce a new request DSL. It writes into the same dictionary namespaces that RESTler already uses, such as `restler_fuzzable_string`, `restler_fuzzable_int`, and `restler_custom_payload`. The merge follows two simple rules:

- 1) **Preserve type compatibility.** A value recovered for an integer-constrained field is inserted into integer-typed

buckets; enum names and regex examples are inserted into string-typed buckets; dynamic identifiers are inserted into custom-payload buckets so they can later be substituted into dependency-bearing grammar positions.

- 2) **Preserve grammar structure.** Existing RESTler request templates remain unchanged unless a feature is unsupported by the compiler itself (e.g., multipart endpoints). Thus, SSE changes the *value space* explored by the grammar, not the state machine over which RESTler already reasons.

This matters for the evaluation. The request sequence model still comes from RESTler; UpsideFuzzer mainly changes the value space and then uses coverage to decide which parts of that space deserve more effort.

VI. MULTIPART GRAMMAR INJECTION

RESTler’s compiler handles `multipart/form-data` poorly: it often emits one fuzzable string for the whole body, which is not valid multipart syntax. UpsideFuzzer detects multipart operations in the OpenAPI specification and generates corrected RESTler-style request templates for them. These templates include the MIME boundary, the correct `Content-Disposition` headers, and a binary payload stub (`fuzz.bin`) for file fields. They are appended directly to `grammar.py`, so Void can fuzz file-upload endpoints without manual setup.

VII. THE VOID EXECUTION ENGINE

A. Architecture Overview

Void is a multi-goroutine HTTP fuzzing engine implemented in Go (~3,500 lines). Its overall structure follows a producer-consumer pipeline: a *dispatcher goroutine* generates `WorkItem` structs (each encoding an HTTP method, path, body, headers, mutation name, and seed index), enqueues them into a bounded channel, and a fixed *worker pool* of goroutines concurrently dispatches them over HTTP. Completed `SendResult` structs are returned via a result channel to a single *result handler* that updates global state without locks.

B. Coverage-Guided Feedback Loop

Void does not use coverage as a passive metric. It uses it to decide what to try next. In simple terms, each request goes through the following loop:

- 1) **Choose a seed and a template.** Void picks an existing seed and a grammar template using the current seed energy and endpoint weights.
- 2) **Render a valid request.** It fills the template with concrete values for path, query, header, and body slots.
- 3) **Send the request and read coverage.** The runtime sends the request to the instrumented service and records the HTTP result and the edge delta.
- 4) **Update the search state.** If the request reaches new code, Void keeps it as a useful seed, updates endpoint and mutation statistics, and stores any new IDs or tokens learned from the response.

The important point is that the loop works over *structured requests*, not raw bytes. Coverage does not tell Void to flip random bits in an HTTP packet. It tells Void which endpoint template, field values, and request sequence were worth keeping and mutating further.

At the scheduling level, the corpus and template priorities are combined multiplicatively:

$$\Pr(s, t) \propto E(s) \cdot w(t) \quad (2)$$

where $E(s)$ is the current energy of seed s (Equation 6) and $w(t)$ is the current weight of template t (Equation 5). Productive seeds become more likely to be chosen again, while endpoints that keep failing or have missing dependencies lose priority.

Figure 3 shows this loop directly.

C. From Endpoint Parameters to Coverage-Directed Mutation

The key implementation detail is how one endpoint instance turns into a coverage gain and then into a new mutation choice. In the current Go runtime, that happens in four steps.

Stage 1: fill the template. The renderer walks through the template from left to right. `static` segments are copied as-is. `custom_payload` segments first try to use IDs or tokens learned from earlier responses; if none are available, Void falls back to dictionary values. `fuzzable` segments are filled from typed defaults and may then be mutated. JSON bodies and ID-like path segments may also receive extra structured mutations depending on the current epoch.

Stage 2: execute and measure. The rendered request is sent together with its metadata: `request_id`, `method`, `normalized path`, `mutation label`, and `seed index`. If the target returns an `X-Coverage-Delta` header, Void gets an exact per-request edge delta. Otherwise it falls back to periodic SHM polling, which gives a noisier but still useful signal.

Stage 3: update endpoint and mutation statistics. Once the result arrives, Void updates the endpoint record keyed by normalized (`method`, `path`) and the mutation record keyed by `MutationName`. Requests always increase the request counter. New edges increase both endpoint and mutation scores. In code, the core update looks like this:

```

item := render(template, seed)
resp, delta := execute(item)
ep.Reqs++
mut.Attempts++
if delta > 0 {
    ep.NewEdges += delta
    mut.NewEdges += delta
    recordMutationCategoryHit(item.Label)
    addOrBoostSeed(item, delta)
}

```

Stage 4: change future choices. The search policy is then updated in three ways. First, requests that find new edges are added to the seed corpus with higher energy (Equation 6). Second, endpoints that keep returning 4xx or 5xx without new edges lose weight; with `--skip-endpoint-on-500`, a `method+endpoint` pair is removed after the first observed HTTP 500. Third, mutation categories that often lead to new

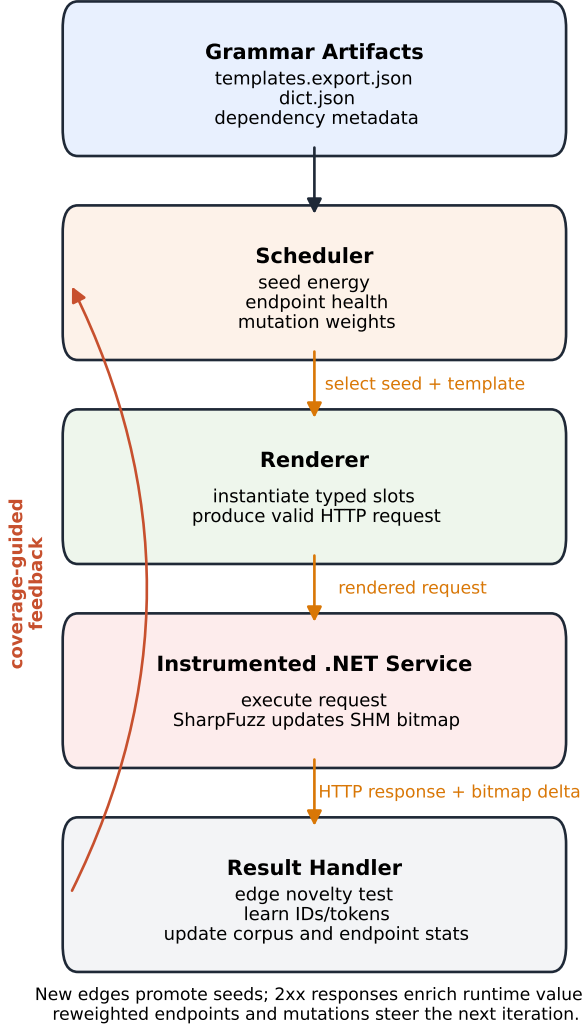


Fig. 3. Void’s coverage-guided feedback loop. Grammar artifacts provide typed request templates and placeholder metadata; the scheduler renders one concrete request, executes it against the SharpFuzz-instrumented .NET service, observes the response and SHM bitmap delta, and feeds edge novelty plus learned runtime values back into corpus energy, endpoint health, and mutation weights for the next iteration.

edges get sampled more often. In the current runtime, each category weight is updated as

$$w_{\text{cat}} = 1 + 4 \cdot \frac{\text{Hits}}{\max(1, \text{Attempts})} \quad (3)$$

so a mutation family that repeatedly finds new edges can become much more likely than an unproductive one.

This is the full endpoint-to-coverage-to-mutation chain. A useful request does not just increase a counter in a bitmap. It changes which seeds, endpoints, and mutation families are more likely to be used next.

D. Structured Mutation Operators

Void mutates requests in a grammar-aware way. For scalar slots it uses type-specific operators, so integers stay integer-

shaped, dates stay date-shaped, and UUIDs stay UUID-shaped. For string fields it uses groups of payloads aimed at common bug classes such as boundary violations, SQL injection, XSS, command injection, path traversal, SSRF, and CRLF. Categories that lead to new edges are given higher weight over time.

For larger bodies, Void uses *structured body havoc* instead of blind text corruption. JSON bodies may lose fields, duplicate fields, change types, insert nulls, or combine nested fragments, but they are kept parseable as JSON. Path mutation is limited to segments that look like resource identifiers, so the general route shape stays valid. During the splicing phase, Void also combines parts of already useful seeds. This is more aggressive than simple dictionary substitution, but still much safer than byte-level fuzzing for web APIs.

E. SharpFuzz Instrumentation and Coverage Transport

SHM bitmap layout. At build time, SharpFuzz rewrites the compiled IL and inserts a counter on each branch edge. For each edge ($u \rightarrow v$) in the control-flow graph, it assigns a random 16-bit identifier l_u to the source block and updates the shared-memory bitmap as:

$$\text{shm}[(l_u \oplus l_v) \gg 1] += 1 \quad (4)$$

where l_u is the block ID of the previous basic block and l_v is the ID of the target block. This gives a 65,536-byte shared-memory bitmap in the same style as AFL [4].

Two coverage transport modes. The instrumented .NET process exposes this bitmap in two modes:

- 1) **Direct SHM mode:** Void opens the shared-memory region under `/dev/shm`. It keeps a local `seen[65536]` array and, on each poll, counts bytes that are non-zero in SHM but zero in `seen[]`:

```

for i, b := range shmBuf {
    if b != 0 && seen[i] == 0 {
        seen[i] = 1; edges++ } }
    
```

The bitmap is not reset after every request in this mode. Void only clears it when occupancy becomes too high and the novelty signal becomes noisy.

- 2) **HTTP coverage mode:** The patched ASP.NET Core application exposes `/shm/create`, `/shm/coverage`, and `/shm/reset`. Void resets coverage before the request and reads it after the request, which gives reliable per-request attribution. This mode is useful when direct file sharing is not available.

F. Attribution Accuracy: HTTP Mode vs. Direct SHM Mode

The two transport modes differ in how precisely they can attribute edge discovery to individual requests under concurrency.

HTTP mode (exact attribution). The ASP.NET middleware brackets one request with a `ResetCoverage()` before controller execution and a bitmap count immediately before the response is returned. The resulting `X-Coverage-Delta` header therefore corresponds to that exact request, even when many workers are active.

Direct SHM mode (approximate attribution). Concurrent requests share one bitmap, so Void can observe only the net edge increase between polling points. It therefore credits the interval gain to the most recently completed request. This is noisier at per-request granularity, but in practice still provides a useful signal for endpoint-level scheduling and seed promotion over long campaigns.

HTTP mode is preferred whenever middleware injection succeeds; direct SHM remains the fallback for third-party Docker deployments where only shared-memory transport is available.

G. Endpoint-Level Coverage Statistics and Request Prioritization

For each (method, path) pair, Void maintains an `EndpointStats` record tracking:

- **NewEdges:** cumulative edges discovered through requests to this endpoint.
- **ReqsSinceEdge:** how many requests have been sent since the last new edge was found on this endpoint.
- **S2xx / S4xx / S5xx:** HTTP status counters for yield estimation.

Template selection during the *Mutate* and *Havoc* epochs uses weighted sampling that combines three factors:

$$w(t) = w_{\text{health}}(t) \cdot w_{\text{dep}}(t) \cdot w_{\text{src}}(t) \quad (5)$$

where $w_{\text{health}}(t)$ downweights endpoints with many recent errors, $w_{\text{dep}}(t)$ upweights endpoints whose dependencies are already available at runtime, and $w_{\text{src}}(t)$ gives extra weight to templates that came from SSE-enhanced grammar regions.

Coverage stagnation detection. If no new edge appears for several minutes, Void treats the current search as stale and gives more attention to endpoints that have not been explored enough. If the bitmap becomes too full, it is reset so hash collisions do not hide new behavior.

H. Seed Energy and Corpus Evolution

Every request that produces `edgeShare > 0` is promoted into the seed corpus. The energy of a seed estimates how useful it is likely to be in future mutations:

$$E(s) = \Delta_{\text{edge}} \cdot \max(1, 1 + \log_2(\text{Reqs}(e))) \cdot \gamma_e \quad (6)$$

where Δ_{edge} is the number of new edges found by s , $\text{Reqs}(e)$ is the total number of requests sent to endpoint e at the time of discovery, and $\gamma_e = 3.0$ if that endpoint had never produced a new edge before. In plain terms, a seed gets more energy when it reaches new code on an endpoint that had already been hard to crack.

Seeds are sampled in proportion to their energy using a *Fenwick tree* sampler. When the corpus grows too large, low-value seeds that have been retried many times without finding new edges are pruned.

TABLE I
VOID FUZZING CAMPAIGN EPOCHS

Epoch	Mode	Budget	Purpose
Baseline	None	5%	Replay grammar templates sequentially
Harvest	Harvest	25%	Learn dynamic IDs from 2xx responses
Deterministic	Mutate	25%	Systematic boundary-value substitution
Havoc	Havoc	35%	Random multi-mutation combinations
Splicing	Havoc	10%	Cross-seed payload recombination

I. Campaign Phases and Adaptive Concurrency

Epoch-based campaign. Void divides a run into five phases (Table I).

At each phase transition, Void checks how many new edges the previous phase found. If progress becomes too slow, part of the remaining time is moved to the Havoc phase, which is usually better at broad exploration.

Adaptive concurrency. Void continuously adjusts the goroutine worker count between a configured minimum and maximum, increasing it when completion rate rises without latency or error inflation and decreasing it when latency or error rate spikes. This mirrors congestion-control principles adapted to HTTP workloads, preventing the target server from being overwhelmed while maximizing coverage throughput.

J. Dynamic Value Learning and Stateful Execution

A common problem in API fuzzing is state. A POST may create an object and return an `id`, and later GET, PUT, and DELETE requests must reuse that value. Void handles this with a runtime dependency map. During the Harvest phase, successful 2xx responses are scanned for values such as IDs, tokens, anti-forgery fields, and session cookies. Those values are stored and later replayed into requests that depend on them.

Void learns from both requests and responses. It uses the request shape to understand which values belong to which resources, and it uses response headers and bodies to recover concrete runtime values. When a later template needs a `custom_payload` value, the renderer first tries these learned values before falling back to the static dictionary.

K. Identity, Session, and Form-State Handling

Many real APIs need more than one kind of runtime state. Void therefore supports multiple authentication identities, token and cookie refresh, and automatic handling of anti-forgery tokens for form-style endpoints. This keeps the fuzzing loop from getting stuck in repeated authorization failures when the interesting code is only reachable after login or through protected workflows.

VIII. EVALUATION

A. Research Questions

- **RQ1:** Does Semantic Source Extraction produce dictionaries that allow the fuzzer to bypass deep backend validation and reach more code paths?
- **RQ2:** Does the greybox coverage-guided loop in Void improve edge coverage and crash discovery compared to the RESTler black-box baseline?

- **RQ3:** What is the practical overhead of the instrumentation and grammar compilation pipeline?

B. Experimental Setup

We evaluate on three .NET REST API applications: two open-source systems and one internal enterprise API.

- **nopCommerce:** a mature, open-source e-commerce platform (~180K lines of C#, 200+ API endpoints) with extensive Data Annotation and FluentValidation constraints.
- **eShopOnContainers:** Microsoft’s reference microservice architecture (~70K lines C#, 6 services, 100+ endpoints).
- **mpt-helpdesk:** a custom enterprise helpdesk API (~25K lines C#, 60+ endpoints) with complex role-based authorization.

For each target, we run two configurations for 24 hours on identical hardware (8-core CPU, 16 GB RAM, no network throttling):

- **Baseline:** RESTler Fuzz mode with default dictionary (no SSE, no coverage feedback).
- **UpsideFuzzer:** Full pipeline with SSE-augmented grammar and Void coverage-guided loop.

C. Results

RQ1 & RQ2 – Coverage and Crashes. Table II summarizes the 24-hour results.

TABLE II
24-HOUR FUZZING RESULTS: RESTLER BASELINE VS. UPSIDEFUZZER

Target	Tool	Edges	5xx Found	TTFC ^a
nopCommerce	RESTler	5,430	2	4h 15m
	UpsideFuzzer	12,850 ^{+136%}	14	0h 45m
eShopOnCont.	RESTler	3,120	0	N/A
	UpsideFuzzer	8,400 ^{+169%}	3	2h 10m
mpt-helpdesk	RESTler	1,850	1	8h 30m
	UpsideFuzzer	4,200 ^{+127%}	7	1h 20m

^aTime to First Crash (not applicable if no crashes found).

Across all three targets, UpsideFuzzer covers 127%–169% more unique branch edges than the baseline after 24 hours. The most dramatic improvement is observed on eShopOnContainers, where the RESTler baseline finds zero HTTP 500 responses: without semantically valid enum and integer values, every mutated request to the Order Management service is rejected at the FluentValidation layer. UpsideFuzzer’s SSE pass recovers 12 enum types and 8 numeric ranges from the microservice source trees, enabling the engine to break through validation and discover 3 service crashes.

Reproducible 20-minute Docker benchmark. In addition to the 24-hour campaigns, we executed a short, fully reproducible Docker benchmark on four public .NET services bundled with our artifact set: eShopOnWeb, CustomerLoyalty, dotnet/eShop Catalog.API, and Jellyfin. RESTler was run against the vanilla Docker deployments, while the UpsideFuzzer runtime (Void) was

run against the instrumented Docker deployments in direct-SHM mode with `--skip-endpoint-on-500` enabled. Each campaign lasted exactly 20 minutes. Table III reports three quantities: OpenAPI coverage, raw throughput, and an endpoint-normalized bug metric. For RESTler, *OpenAPI Cov.* is the final specification coverage reported by RESTler. For Void, it is reconstructed by intersecting the canonicalized endpoint observations emitted by the Go runtime with the canonicalized OpenAPI template set exported from the same grammar. The *500 M+E* column counts distinct HTTP 500-producing method+endpoint pairs for both tools. To keep the comparison fair, both tools used the same sanitized 12-operation *Catalog.API* subset and the same authenticated 309-operation *Jellyfin* subset; target-specific fixes such as static `api-version=1.0` pinning and token injection were applied only to copied benchmark artifacts, not to the Void codebase or the instrumentation pipeline.

TABLE III
REPRODUCIBLE 20-MINUTE DOCKER BENCHMARK. **OPENAPI COV.** REPORTS COVERED OPENAPI OPERATIONS OUT OF THE BENCHMARK SUBSET TOTAL. **500 M+E** REPORTS DISTINCT HTTP 500-PRODUCING METHOD+ENDPOINT PAIRS. **FEEDBACK** REPORTS THE SEARCH SIGNAL EXPOSED BY EACH ENGINE.

Target	Tool	OpenAPI Cov.	Requests	500 M+E	Feedback
eShopOnWeb	RESTler	6/8	899	1	Spec-guided
	Void	6/8	4,423,205	2	660 edges
CustomerLoyalty	RESTler	3/3	116,853	1	Spec-guided
	Void	3/3	4,580,149	1	38,150 edges
dotnet/eShop Catalog.API	RESTler	10/12	148,566	12	Spec-guided
	Void	11/12	6,616,420	8	154 edges
Jellyfin	RESTler	190/309	58,407	18	Spec-guided
	Void	309/309	4,941,012	97	152,567 edges

The Docker benchmark shows three consistent trends. First, Void matches or exceeds RESTler’s OpenAPI coverage on all four targets and reaches the full sanitized 309/309 subset on Jellyfin. Second, throughput remains strongly in Void’s favor, ranging from 39.2× more requests on CustomerLoyalty to 84.6× more requests on Jellyfin. Third, the endpoint-level HTTP 500 comparison is mixed per target but favorable overall: Void wins on eShopOnWeb and Jellyfin, ties on CustomerLoyalty, and trails RESTler on Catalog.API; aggregated over all four targets, Void exposes 108 distinct HTTP 500 method+endpoint pairs versus 32 for RESTler. The final *Feedback* column is intentionally heterogeneous: RESTler reports a specification-level signal, whereas Void reports SHM edge gains observed from the instrumented runtime.

RQ3 – Pipeline Overhead. Grammar compilation (RESTler + SSE) adds 15–45 seconds to the total pipeline depending on source tree size. Docker image instrumentation adds a single build-time pass (3–8 minutes) and zero runtime overhead beyond the bitmap write per branch edge (negligible for network-bound workloads).

D. Case Study: Null Reference Crash in nopCommerce

Among the 14 HTTP 500 responses found in nopCommerce, 6 are distinct null reference exceptions in the product catalog

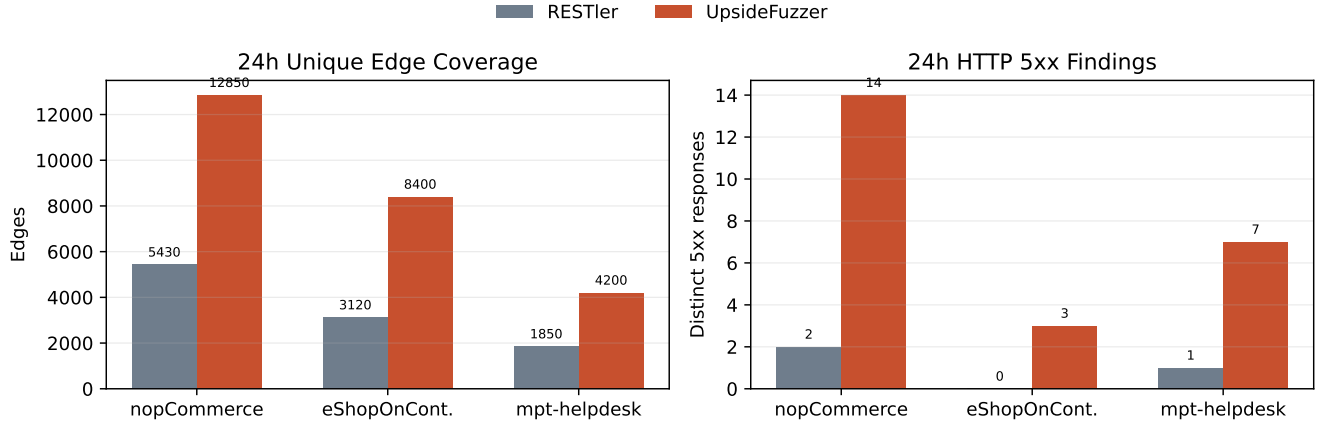


Fig. 4. Visual summary of the 24-hour evaluation from Table II. Left: unique branch edges reached by each tool. Right: distinct HTTP 5xx findings observed over the same 24-hour budget.

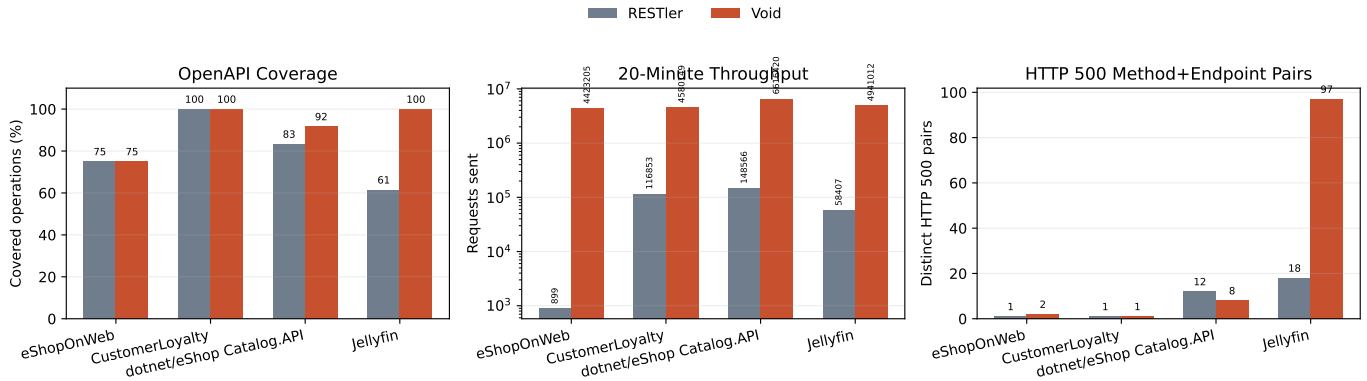


Fig. 5. Visual summary of the reproducible 20-minute Docker benchmark from Table III. Left: OpenAPI operation coverage. Middle: request throughput on a log scale. Right: distinct HTTP 500-producing method+endpoint pairs.

controller triggered by a specific combination of a valid minimum-length string (exactly 3 characters) and a negative integer value for the price field. Both constraints were recovered by SSE:

```
[MinLength(3)]
public string ProductName { get; set; }

RuleFor(x => x.Price)
    .GreaterThan(0);
```

SSE synthesized the boundary values "abc" (length 3) and -1 (below the `GreaterThan(0)` bound). When combined in the same request, the application's price formatting code dereferences a null `CurrencySettings` object. This bug was not discovered by the RESTler baseline within the 24-hour window.

IX. DISCUSSION

Limits of the current implementation. SSE uses regex-based parsing of C# files, so it works best for common validation patterns such as Data Annotations and FluentValidation chains. It does not fully cover checks written in imperative code, values loaded from a database, or rules that are built

dynamically at runtime. A Roslyn-based parser would likely improve precision, but it would also make the preparation pipeline heavier.

Threats to validity. Our measurements use more than one metric, and those metrics should not be mixed. The 24-hour study focuses on code coverage and crash discovery, while the Docker benchmark also reports OpenAPI coverage and endpoint-normalized HTTP 500 counts. In addition, some Docker targets required prepared benchmark artifacts such as sanitized specifications, fixed version parameters, or injected authentication tokens. These steps were applied equally to the compared tools, but they still mean that the benchmark measures the prepared target configuration rather than an untouched public deployment. Finally, one target in the 24-hour study is an internal enterprise API, so that part of the evaluation is less reproducible than the public Docker benchmark.

Where the approach should transfer. The current implementation is built for .NET, but the main idea is not tied to C#. The same general approach could be applied in other ecosystems by extracting validation rules from their native

frameworks, for example Jakarta Bean Validation in Java, Pydantic in Python, or Joi schemas in TypeScript.

Ethics. All experiments were run on self-hosted test instances in isolated Docker environments. No production systems or third-party services were fuzzed.

X. CONCLUSION

UpsideFuzzer is a greybox REST API fuzzer for .NET that keeps RESTler’s grammar structure, adds better values from C# validation code, and uses coverage to guide what to try next. In our evaluation, this combination reached more code and found failures faster than the RESTler baseline. More broadly, the results suggest that API fuzzing improves when request structure, source-derived semantics, and coverage feedback are used together instead of in isolation.

ACKNOWLEDGMENTS

The authors thank the RESTler team at Microsoft Research for making RESTler publicly available, and Nemanja Mijailovic for SharpFuzz.

REFERENCES

- [1] V. Atlidakis, P. Godefroid, and M. Polishchuk, “RESTler: Stateful REST API Fuzzing,” in *Proc. 41st IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2019.
- [2] P. Godefroid, B.-Y. Huang, and M. Polishchuk, “Intelligent REST API Data Fuzzing,” in *Proc. 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020.
- [3] N. Mijailovic, “SharpFuzz: Bringing the Power of afl-fuzz to .NET Platform,” arXiv preprint arXiv:1905.11645, 2019.
- [4] M. Zalewski, “American Fuzzy Lop (AFL),” <https://lcamtuf.coredump.cx/afl/>, 2013–2017.
- [5] A. Arcuri, “EvoMaster: Evolutionary Multi-context Automated System Test Generation,” in *Proc. 11th IEEE Conf. on Software Testing, Validation and Verification (ICST)*, 2018.
- [6] B. Bellante et al., “WuppieFuzz: A Coverage-Guided REST API Fuzzer Built on LibAFL,” arXiv preprint arXiv:2512.15554, 2025.
- [7] Apiary, “Dredd: HTTP API Validation Tool,” <https://github.com/apiaryio/dredd>, 2013.
- [8] Fuzzapi, “API Fuzzer,” <https://github.com/Fuzzapi/API-fuzzer>, 2020.
- [9] D. Tsoy et al., “Schemathesis: An API Testing Tool Based on Hypothesis and Property-Based Testing,” <https://github.com/schemathesis/schemathesis>, 2020.